

Clothing Exchange & Swap Marketplace

Detailed Project Report - Internship Submission

Project Type	Web Application / Internship Project
Frontend	Angular, HTML, SCSS, Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB with Mongoose
Authentication	JWT-based authentication
File Upload	Multer image upload
Real-time Communication	Socket.IO
Primary Function	Discover clothing, exchange items, manage swaps, chat

Project summary: The application provides a digital marketplace where users can list clothing items, browse available listings, request swaps using their own listed items, manage swap status, and communicate in real time with the other participant.

1. Introduction

Clothing Exchange & Swap Marketplace is a full-stack web application designed to make clothing exchange easier, more organized, and more interactive. Instead of relying on informal messaging or unstructured social posts, users can publish clothing listings, discover items, submit a structured swap request, manage the request lifecycle, and communicate directly through an integrated real-time chat.

The project follows a modular client-server architecture. Angular is used for the user interface, Node.js and Express.js provide the REST API layer, MongoDB stores application data, and Socket.IO provides real-time communication for swap conversations.

2. Problem Statement

- Users need a simple way to find clothing items suitable for exchange.
- Existing informal exchange workflows do not provide structured swap requests or status tracking.
- Users need to know which item they are offering and which item they are requesting.
- Communication between swap participants should be attached to the swap context.
- A marketplace needs authentication, ownership validation, and secure access to user-specific functions.

3. Objectives

- Build a working web marketplace for clothing exchange.
- Allow authenticated users to create, update, and manage their own clothing listings.
- Provide search and filtering for marketplace discovery.
- Implement secure swap-request validation and duplicate pending-request prevention.
- Provide dashboard-based tracking of sent and received swap requests.
- Support accept, reject, cancel, and complete swap states.
- Provide real-time two-user chat with persistent message history.
- Provide an administrative marketplace overview for listings.

4. Scope of the Implemented Project

The internship version focuses on the core marketplace and exchange workflow. Implemented functionality includes authentication, marketplace listings, item details, image upload, swap requests, dashboard tracking, swap status changes, Socket.IO chat, persistent chat messages, and an administrative listing overview.

Advanced capabilities such as AI styling, payment processing, delivery-address management, and full CRUD moderation APIs are not part of the implemented submission scope.

5. Technology Stack

Frontend Technology

- Angular standalone components for modular UI development.
- TypeScript for typed client-side application logic.
- HTML and SCSS for page structure and styling.

- Tailwind CSS for responsive utility-based styling where used.

Backend Technology

- Node.js runtime.
- Express.js REST API.
- Mongoose ODM for MongoDB interaction.
- JWT for authentication tokens.
- Multer for image uploads.
- Socket.IO for real-time chat.

6. System Architecture

The application uses a three-layer architecture:

- Presentation layer: Angular components, services, routing, forms, and responsive UI.
- Application/API layer: Express routes, controllers, authentication middleware, upload middleware, and Socket.IO event handlers.
- Data layer: MongoDB collections accessed through Mongoose models.

High-level flow

```
Angular Client
|
| HTTP / REST + Authorization header
v
Express / Node.js API
|---- JWT authentication
|---- Clothing Item APIs
|---- Swap APIs
|---- Upload handling
|
+---- MongoDB / Mongoose

Angular Chat <==== Socket.IO ====> Node.js Socket.IO server
```

7. Main Functional Modules

7.1 Authentication

Users can register and log in using the authentication API. Successful login returns a JWT token. The frontend stores authentication information locally and sends the token through the Authorization Bearer header for protected API calls.

7.2 Marketplace

The marketplace exposes clothing listings and supports discovery using filters such as category, gender, size, brand, condition, color, and text search across listing attributes. Listings are displayed with item information and image support.

7.3 Clothing Listing Management

Authenticated users can create listings containing title, description, category, size, brand, brand website, condition, gender, color, location, estimated value, image URL, and ownership information. Users can retrieve their own listings, update them, and delete them.

7.4 Image Upload

Multer is configured to store clothing images under the server uploads directory. Supported formats include JPG, JPEG, PNG, and WEBP. Upload size is limited to 5 MB per image.

7.5 Swap Request

A user can select another user's marketplace item and choose one of their own listed items as the offer. The backend validates both item IDs, verifies ownership, prevents requesting one's own item, prevents offering the same requested item, and prevents duplicate pending requests for the same requested item.

7.6 Dashboard / My Swaps

The dashboard retrieves swaps where the logged-in user is either requester or owner. It presents swap status and item information and provides controls based on the user's role. Owners can accept or reject pending requests; requesters can cancel pending requests; accepted swaps can be marked completed.

7.7 Real-time Chat

Socket.IO creates a room for each swap request. Only participants associated with the swap are permitted to exchange messages. Messages are stored in the swap document, so chat history remains available after refresh.

7.8 Admin Overview

The implemented admin panel provides a marketplace listing overview with search, category filtering, listing status presentation, and summary information. It is an overview interface rather than a full moderation CRUD backend.

8. Database Design

User collection

User records are used for identity, authentication, and role information. Swap and listing records reference users through MongoDB ObjectId relationships.

ClothingItem collection

```
title
description
category
size
brand
brandWebsite
condition
gender
color
location
estimatedValue
imageUrl
owner
createdAt / updatedAt
```

Swap collection

```
requester
owner
requestedItem
offeredItem
message
status
messages[]
createdAt / updatedAt
```

Each chat message stores senderId, messageText, and timestamp. Swap statuses are pending, accepted, rejected, cancelled, and completed.

9. API Design

Clothing Item APIs

```
GET /api/items
GET /api/items/:itemId
GET /api/items/mine
POST /api/items
PUT /api/items/:itemId
DELETE /api/items/:itemId
```

Swap APIs

```
POST /api/swaps
GET /api/swaps/user/:userId
GET /api/swaps/:swapId
PUT /api/swaps/:swapId
```

Protected routes use JWT authentication middleware. The backend also performs participant and ownership checks before sensitive operations.

10. Security and Validation

- Bearer token validation is applied to protected APIs.
- JWT payload supports userId/id/_id variants and resolves the current user from MongoDB.
- Passwords are not returned in authenticated user lookups.
- Swap creation validates ObjectIds before database operations.
- Users can only offer clothing items they own.
- Users cannot request their own item.
- Duplicate pending requests are rejected with a conflict response.
- Image upload accepts only supported image MIME types and enforces a file-size limit.
- Chat sender authorization verifies that the sender is a swap participant.

11. User Workflow

1. Register / Login
2. Browse Marketplace
3. Open Clothing Item
4. Select Request Swap
5. Select an owned item to offer
6. Submit Swap Request
7. Owner reviews the request
8. Owner accepts or rejects
9. Participants open Chat
10. Exchange is completed and status is updated

12. Frontend Component Structure

```
src/app/components/  
admin/  
chat/  
dashboard/  
item-detail/  
list-item/  
login/  
marketplace/  
swap-request/
```

```
src/app/services/  
auth  
item  
swap  
  
src/app/interceptors/  
auth interceptor
```

13. Backend Structure

```
server/  
controllers/  
authController.js  
itemController.js  
swapController.js  
middleware/  
authMiddleware.js  
uploadMiddleware.js  
models/  
ClothingItem.js  
Swap.js  
User.js  
routes/  
authRoutes.js  
itemRoutes.js  
swapRoutes.js  
uploads/  
items/  
server.js
```

14. Testing and Verification

- Backend server was verified on port 5000 with successful MongoDB connectivity.
- Marketplace listing retrieval was tested.
- Authentication and protected requests were tested with real user accounts.
- Swap request creation was verified end-to-end and successfully persisted a swap record in MongoDB.
- Socket.IO connection, room joining, and two-user message delivery were tested.
- Chat messages were implemented as persistent swap messages.
- Dashboard functionality was updated to display sent and received swaps and status actions.
- Admin listing overview was implemented against the existing marketplace listing API.

15. Example Verified Swap Flow

During development, a valid swap was created between two test users. The requester offered their own clothing item in exchange for a clothing item owned by the other participant. Backend logs confirmed successful validation and persistence of the swap.

```
Request payload  
requestedItem: 6aabaefd704361c24cf77352  
offeredItem: 6aabaea8704361c24cf77351  
message: ""  
  
Result  
SWAP CREATED: 6aab577c1e46349a5b950c2
```

16. Current Limitations

- The current internship version does not include a delivery-address module.
- No online payment gateway is implemented.
- Admin moderation is an overview interface; dedicated admin CRUD APIs are not part of the current scope.
- AI-powered styling/recommendation functionality is outside the implemented submission scope.
- Deployment configuration depends on the target hosting environment and database credentials.

17. Future Scope

- Add complete user profile and delivery address management.
- Add advanced notification workflows for swap events.
- Add AI-based style matching and clothing recommendations.
- Add stronger admin moderation tools and audit history.
- Add deployment, monitoring, analytics, and production security hardening.
- Add location-aware exchange options and optional delivery integrations.

18. Deployment Preparation

For deployment, the Angular frontend should be built for production and hosted on a static/web hosting platform, while the Node.js API should be deployed on a server platform with MongoDB configured as the production database. Environment variables such as JWT secrets and database credentials must remain outside source control.

```
Frontend build
ng build

Backend start
npm run dev

Database
MongoDB / clothing_exchange_marketplace
```

19. Conclusion

The Clothing Exchange & Swap Marketplace successfully implements the central exchange workflow required for an internship-ready full-stack project. The system combines marketplace discovery, authenticated listing management, structured swap requests, role-based swap status actions, persistent data storage, image uploads, and real-time participant communication. The resulting application demonstrates practical use of modern Angular and Node.js development patterns together with MongoDB, JWT, Multer, and Socket.IO.

Project Status: Internship submission MVP completed according to the implemented project scope.